Jasper: Engineering a Global Workspace for Latent Reasoning in Hybrid State-Space Models

Jasper Research Project

July 2026

Abstract

Recent work from Anthropic has revealed that language models develop a compact, privileged internal workspace—the J-SPACE—that causally carries multi-step reasoning, while Samsung's Tiny Recursive Model (TRM) has demonstrated that iterative refinement through a tiny recurrent network can substitute for parameter count on hard reasoning tasks. We present Jasper, an architecture that combines these two insights into a single system: a hybrid Mamba-2/attention backbone with an explicitly engineered workspace module and a recurrent core that iterates on the workspace state. The key architectural observation is that Mamba-2's selective state-space mechanism already maintains a fixed-size compressed state that is functionally shaped like the J-SPACE—a coincidence that makes the combination more powerful than applying the same ideas to a pure transformer. We argue that this approach is a direct and more efficient alternative to chain-of-thought "thinking tokens": it reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and allows trading inference-time compute depth for parameter count. We describe a proof-of-concept experiment that trains four parameter-matched model variants on synthetic reasoning tasks with controllable depth, and outline the pre-registered criteria for a go/no-go decision on scaling.

Contents

1 Introduction

Large language models have demonstrated impressive reasoning capabilities, but the dominant paradigm for scaling reasoning—generating long chains of "thinking tokens" before producing an answer—has fundamental inefficiencies. Each thinking token requires a full forward pass through the model, the key-value cache grows linearly with the length of the reasoning trace, and the entire chain must be serialized in token space, consuming both memory and latency. On constrained deployment targets such as web browsers or mobile devices, this memory growth is the binding constraint.

Two recent findings suggest an alternative path. First, Anthropic's discovery of the J-SPACE (Anthropic, 2026) revealed that the reasoning-critical machinery inside a language model is already small: a compact set of verbalizable representations, occupying less than 10% of activation variance, carries the causal bulk of multi-step reasoning. This workspace is not the full residual stream—it is a privileged, broadcasted subset that functions analogously to the global workspace in cognitive science (Baars, 2005). Second, Samsung's Tiny Recursive Model (TRM) (Jolicoeur-Martineau, 2025) and Geiping et al.'s recurrent-depth transformers (Geiping et al., 2025) have shown that iterative refinement through a small recurrent network can substitute for parameter count: a 7M-parameter model recursing on itself can outperform models with 1000× more parameters on reasoning benchmarks.

Jasper combines these insights. We engineer an explicit workspace module—a compact set of learned slot vectors that serve as a persistent scratchpad—and pair it with a recurrent core that iterates on the workspace state. The backbone is a hybrid Mamba-2/attention model, which brings a third observation into play: Mamba-2's selective state-space duality (SSD) (Dao and Gu, 2024) maintains a fixed-size compressed state that is functionally shaped like the J-SPACE. This is an unlikely but convenient coincidence: the architecture already has a mechanism that resembles the global workspace, so the engineered workspace and the SSM state reinforce each other rather than competing for the same functional role.

The result is a model that reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and can trade inference-time compute (more loop iterations) for parameter count. We argue that this is a direct alternative to thinking tokens—one that is more memory-efficient, faster, and potentially more capable of reasoning that is not easily expressed in words.

2 Background

2.1 The J-Space: A Global Workspace in Language Models

Anthropic (2026) introduced the Jacobian lens (J-lens), an interpretability technique that identifies which concepts a language model is *poised to verbalize* at any point in its processing. Applying the J-lens across layers reveals a set of representations—the J-SPACE—that exhibits the functional properties of a global workspace (Dehaene, 2014; Baars, 2005):

- **Verbal report:** J-space contents can be read out as words the model could say.
- **Directed modulation:** Instructing the model to think about a concept causes that concept to appear in the J-space.
- **Silent reasoning:** Intermediate steps of multi-step problems appear in the J-space before the model verbalizes them.
- **Selective causality:** Ablating the J-space destroys multi-step reasoning while leaving single-hop recall intact.
- **Compactness:** The J-space carries fewer than ∼25 concepts at a time and occupies <10% of activation variance.

Crucially, the J-SPACE was not designed—it emerged during training. But its properties are

exactly what one would want from an engineered reasoning workspace: it is small, privileged, broadcasted, and causally responsible for multi-step reasoning. This raises the question: *can we engineer such a workspace explicitly, and does doing so improve reasoning in a small model?*

2.2 Iterative Reasoning with Tiny Networks

Jolicoeur-Martineau (2025) introduced the Tiny Recursive Model (TRM), a 7M-parameter network that recurses on itself to iteratively refine answers. TRM alternates between a "think" step (updating a latent scratchpad $\mathbf{z}$) and an "act" step (updating a solution embedding $\mathbf{y}$), unrolled for up to 16 iterations with deep supervision. Despite having less than 0.01% of the parameters of frontier LLMs, TRM achieves 45% accuracy on ARC-AGI-1 and 8% on ARC-AGI-2, outperforming models like DeepSeek R1 and o3-mini on these benchmarks.

Independently, Geiping et al. (2025) demonstrated that recurrent-depth transformers—which iterate a transformer block for a variable number of steps at inference time—can scale test-time compute in latent space. Their 3.5B-parameter model, trained on 800B tokens, improves performance on reasoning benchmarks with additional inference-time iterations, sometimes matching the performance of a 50B-parameter model.

Both works share a key insight: *depth (iterations) can substitute for width (parameters)*. But both use attention-based cores, which means the key-value cache grows with sequence length, and the recurrent state is carried in the residual stream—a high-dimensional, unstructured representation.

2.3 Mamba-2 and State Space Duality

Dao and Gu (2024) established the state space duality (SSD) framework, showing that Mamba's selective state-space model is closely related to attention through a shared mathematical structure. The Mamba-2 layer computes:

$$y_t = \sum_{s \le t} \underbrace{\exp\Big(-\sum_{k=s+1}^{t} a_k\Big)}_{\text{decay}} \cdot (C_t^\top B_s) \cdot V_s \tag{1}$$

where B_s, C_t are input-dependent projections, a_k is a scalar decay, and V_s is the value vector. This can be viewed as a form of attention with a learned, position-dependent decay mask—the "selective scan."

The key property for our purposes is that the SSM maintains a **fixed-size state** $\mathbf{h}_t$ that is updated at each position:

$$\mathbf{h}_t = \bar{A}_t \mathbf{h}_{t-1} + \bar{B}_t x_t \tag{2}$$

where $\bar{A}_t, \bar{B}_t$ are discretized state transition matrices. This state is **compressed**: it has a fixed dimension d_{state} regardless of sequence length, and it is **selective**: the input-dependent gates control what information is written and what is forgotten. This is functionally analogous to the J-SPACE—a small, privileged state that carries task-relevant information forward through the computation.

3 The Mamba-2 Coincidence

The central architectural observation behind Jasper is what we call the *Mamba-2 coincidence*: the selective state-space mechanism already maintains a representation with the functional shape of the J-SPACE, without any engineering. Consider the parallels:

This parallel is not merely metaphorical. When we build a hybrid model with Mamba-2 layers and an explicit workspace module, the SSM's compressed state and the workspace slots

Property	J-space (Anthropic)	Mamba-2 SSM State
Fixed size	∼25 concepts, <10% of activation variance	$d_{\text{state}} = 64$ per head, regardless of sequence length
Selective	Broadcasted more widely than other representations	Input-dependent gates control write/forget
Persistent	Carries intermediate reasoning steps across layers	Carries information across the sequence
Causally responsible	Ablation destroys multi-step reasoning	Removing state connections breaks long-range dependencies
Emerges in training	Not designed, emerged in Claude	Not designed for reasoning, emerged from language modeling

Table 1: Functional parallel between the J-SPACE and Mamba-2's SSM state.

reinforce each other: the SSM state provides a natural substrate for the workspace to read from and write to, and the workspace provides a structured, interpretable interface to the SSM's otherwise opaque state. In a pure transformer, the workspace must carve out its role from the residual stream against the competing pressure of attention's global mixing. In a hybrid Mamba model, the workspace aligns with an existing architectural feature.

The coincidence becomes powerful when combined with a recurrent core. In a recurrent transformer (Geiping et al., 2025), each iteration processes the full sequence with attention, growing the KV cache. In a recurrent Mamba model, each iteration updates the SSM state in-place—memory is flat in both sequence length *and* loop count. The workspace state, carried between iterations, provides a stable anchor for iterative refinement without any memory growth.

4 Architecture

4.1 Overview

The Jasper architecture combines four components:

1. **Mamba-2 layers** (SSD form) for sequence processing with flat memory.
2. **Attention layers** at selected positions for exact retrieval capability.
3. **Workspace module** — a perceiver-style cross-attention block with learned slot vectors.
4. **Recurrent core** — a range of layers that are iterated K times, with the workspace state carried between iterations.

4.2 Workspace Module

The workspace module is an explicit implementation of the J-SPACE concept. It maintains $m = 16$ learned slot vectors $\mathbf{S} \in \mathbb{R}^{m \times d}$, where $d = 384$ is the model dimension. At each application, it performs two phases:

Read phase (sequence → workspace). Slots attend over the hidden states $\mathbf{X} \in \mathbb{R}^{T \times d}$:

$$\mathbf{Q}_s = \mathbf{S}\, W_Q, \quad \mathbf{K}_x = \mathbf{X}\, W_K, \quad \mathbf{V}_x = \mathbf{X}\, W_V \tag{3}$$

$$\mathbf{A}_{\text{read}} = \text{softmax}\left(\frac{\mathbf{Q}_s \mathbf{K}_x^\top}{\sqrt{d_h}}\right) \tag{4}$$

$$\mathbf{S}' = \text{SlotNorm}(\mathbf{S} + \sigma(g_r) \cdot \mathbf{A}_{\text{read}} \mathbf{V}_x\, W_O) \tag{5}$$

where $d_h = d/n_{\text{heads}}$ is the per-head dimension, σ is the sigmoid function, and g_r is a learned scalar gate. The sigmoid gating allows the model to learn how much of the sequence to absorb into the workspace at each step.

Write phase (workspace → sequence). Hidden states attend over the updated slots:

$$\mathbf{Q}_x = \mathbf{X}\, W'_Q, \quad \mathbf{K}_s = \mathbf{S}'\, W'_K, \quad \mathbf{V}_s = \mathbf{S}'\, W'_V \tag{6}$$

$$\mathbf{A}_{\text{write}} = \text{softmax}\left(\frac{\mathbf{Q}_x \mathbf{K}_s^\top}{\sqrt{d_h}}\right) \tag{7}$$

$$\mathbf{X}' = \text{RMSNorm}(\mathbf{X} + \sigma(g_w) \cdot \mathbf{A}_{\text{write}} \mathbf{V}_s\, W'_O) \tag{8}$$

The slot state $\mathbf{S}'$ is persistent: it is carried forward to the next workspace application and, in the recurrent core, across loop iterations. SlotNorm (RMSNorm applied to the slot dimension) prevents unbounded growth of slot activations across iterations.

Slot parameter normalization. The learned slot parameters $\mathbf{S}$ are subject to a positive feedback loop: large slots produce sharp attention distributions, which produce large gradients, which cause the slots to grow further. To break this loop, the slot parameters are normalized to unit RMS after each optimizer step:

$$\mathbf{S} \leftarrow \frac{\mathbf{S}}{\text{RMS}(\mathbf{S})}, \quad \text{RMS}(\mathbf{S}) = \sqrt{\frac{1}{md}\sum_{i,j} S_{ij}^2} \tag{9}$$

This is a parameter constraint (analogous to weight clipping in GANs) rather than a learned normalization. It allows the *direction* of each slot vector to change freely while constraining its magnitude, preventing the runaway growth that causes gradient explosion.

Spectral normalization of workspace weights. Even with slot normalization, the attention weight matrices (W_Q, W_K, W_V, W_O) can grow during training, producing large attention logits that create sharp softmax distributions. To address this root cause directly, we cap the spectral norm of all eight workspace weight matrices at a bound C after each optimizer step:

$$W \leftarrow W \cdot \min\left(1,\ \frac{C}{\sigma_{\max}(W)}\right) \tag{10}$$

where $\sigma_{\max}(W)$ is the largest singular value, estimated via 10 steps of power iteration on-device. This is the technique that stabilized GAN discriminators Miyato et al. (2018), which face the same problem: unbounded weights → sharp distributions → gradient instability. The cap (rather than a hard normalization to C) lets the optimizer grow weights naturally up to the bound, then prevents further growth — breaking the positive feedback loop without disrupting training.

With $C = 5$, unit-RMS slots, and LayerNorm on the input, the attention logits are bounded by:

$$|\text{logit}_{ij}| \leq \|W_Q\|_2 \|W_K\|_2 \|\text{slots}\| \|x\| / \sqrt{d_h} \leq C^2/\sqrt{d_h} \approx 2.55 \tag{11}$$

giving a maximum attention ratio of $e^{2\times 2.55} \approx 164{:}1$ — sufficient for selective attention, but preventing the e^{100+} ratios that cause gradient explosion. The model can still learn the *direction* of each weight matrix (which determines what it attends to) and grow its magnitude up to C (which determines attention sharpness), but cannot grow beyond C.

Design rationale. The workspace is deliberately small ($m = 16$ slots, ∼2M parameters total) to mirror the compactness of the J-SPACE. The two-phase design (read then write) mirrors the global workspace theory's broadcast cycle: information is first consolidated into the workspace, then broadcast back to the processing stream.

4.3 Recurrent Core

Layers c_{start} through $c_{\text{end}} - 1$ (by default, layers 6–9) form the recurrent core. These layers are applied K times in sequence, with the workspace state carried between iterations:

$$\mathbf{x}^{(k+1)}, \mathbf{S}^{(k+1)} = \text{CoreLayers}\left(\mathbf{x}^{(k)}, \mathbf{S}^{(k)}\right), \quad k = 0, \ldots, K-1 \tag{12}$$

During training, K is sampled uniformly from $\{1, \ldots, K_{\max}\}$ for each batch. For hardware compilation (e.g., on Tenstorrent accelerators), the loop is always unrolled to $K_{\max}$, with iterations beyond the sampled K masked to identity via a blending operation:

$$\mathbf{x}^{(k+1)} = \alpha_k \cdot \mathbf{x}^{(k)}_{\text{new}} + (1 - \alpha_k) \cdot \mathbf{x}^{(k)} \tag{13}$$

where $\alpha_k = 1$ if $k < K$ and $\alpha_k = 0$ otherwise. This keeps the computation graph static while allowing variable-depth training.

$1/\sqrt{K}$ residual scaling. When the core layers are shared across K iterations, gradients from all iterations accumulate on the shared parameters. Without normalization, the accumulated gradient norm scales as $\sim \sigma\sqrt{K}$, causing instability at large K. To normalize this, the blend factor is scaled by $1/\sqrt{K}$:

$$\alpha_k = \frac{\mathbb{1}[k < K]}{\sqrt{K}} \tag{14}$$

This turns the full replacement ($\alpha_k = 1$) into a partial update, where each iteration contributes $1/\sqrt{K}$ of the new state. The total variance added to the signal after K iterations is then independent of K, following the variance-preserving initialization principle used in DeepNorm (Wang et al., 2022). The scaling uses the actual K sampled for each batch, so batches with different K values produce gradients of comparable magnitude.

At inference time, K can be chosen freely—this is the test-time compute scaling mechanism. Harder problems can be allocated more iterations, directly trading compute for accuracy.

4.4 Hybrid Backbone

The full model uses 13–14 layers total, with attention inserted at positions 5 and 10. The attention layers provide exact retrieval capability—a known weakness of pure SSMs (Arora et al., 2024)—while the Mamba-2 layers provide efficient sequence processing with flat memory. The hybrid ratio (∼85% SSM, ∼15% attention) follows the empirical finding that a minority of attention layers recovers most of the retrieval gap while preserving the throughput and memory advantages of SSMs.

5 Comparison to Thinking Tokens

The dominant approach to scaling reasoning in language models is chain-of-thought (CoT) prompting: the model generates a sequence of intermediate "thinking tokens" before producing its final answer. This approach, while effective, has several structural inefficiencies that the workspace-recurrent architecture addresses directly.

The key distinction is that thinking tokens reason in *token space*—the model must verbalize each intermediate step, serialize it, and re-process it on the next forward pass. This is slow (one forward pass per token), memory-hungry (the KV cache grows with each token), and constrains reasoning to what can be expressed in language. As Geiping et al. (2025) note, "a substantial amount of thought happens through complex, recurrent firing patterns in the brain, before the first word of an answer is uttered."

Workspace recurrence reasons in *latent space*—the workspace state is updated in-place through matrix operations, without generating any tokens. This is faster (one forward pass per

Property	Thinking Tokens (CoT)	Workspace Recurrence (Jasper)
Reasoning medium	Token space (discrete, serial)	Latent space (continuous, parallel)
Memory growth	Linear in reasoning length (KV cache)	Flat (fixed workspace + SSM state)
Tokens generated	One per reasoning step	Zero—reasoning is internal
Legibility	Fully legible (readable text)	Partially legible (workspace probes)
Compute control	Number of generated tokens	Number of loop iterations K
Training data	Requires CoT supervision or RL	Standard next-token prediction + deep supervision
Expressive range	Limited to verbalizable reasoning	Can capture non-verbal reasoning patterns
Deployment cost	High (long generation, large cache)	Low (short output, fixed memory)

Table 2: Comparison of thinking tokens vs. workspace recurrence.

iteration, not per token), memory-flat (the workspace has a fixed size regardless of reasoning depth), and can capture reasoning patterns that are not easily expressed in words. The trade-off is legibility: thinking tokens produce readable traces, while workspace reasoning is only partially legible through probes (see Section 6).

For deployment on constrained devices (web browsers, mobile phones), the memory advantage is decisive. A model that generates 1000 thinking tokens accumulates a KV cache of $\sim$1000 $\times$ d_{model} $\times$ n_{layers} floats. A model that iterates its workspace 100 times uses the same fixed workspace state ($m \times d$ floats) plus the SSM state ($d_{\text{state}} \times d$ floats), regardless of K.

6 Experimental Design

6.1 Ablation Ladder

We train four parameter-matched model variants ($\sim$25–30M parameters each) that form an ablation ladder—each cell adds one component, so the marginal contribution of each is isolated:

Cell	Architecture	What it tests
A	Pure Mamba-2 (14 layers)	SSM floor; establishes baseline, especially on state-tracking where SSMs are known to struggle
B	Hybrid (12 Mamba-2 + 2 attention)	The real baseline the workspace must beat; hybridization recovers retrieval capability
C	Hybrid + workspace (13 layers + 16-slot workspace)	Does an engineered workspace help without recurrence?
D	Hybrid + workspace + recurrent core (layers 6–9 looped K times)	The full architecture; does recurrence + workspace beat the hybrid baseline?

Table 3: The four-cell ablation ladder. All cells target $\sim$30M parameters.

A fifth cell (E) serves as a learning-rate control: identical architecture to Cell B but with Cell C/D's lower learning rate, to distinguish the workspace effect from the learning-rate effect.

6.2 Training Configuration

All cells use AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}$) with weight decay 0.1 and gradient clipping at norm 1.0. The learning rate follows a linear warmup schedule over 200 steps (for C/D/E) or 1500 steps (for A/B), followed by a constant peak. Cells A and B use peak lr=6×10^{-4}; Cells C, D, and E use peak lr=2×10^{-4}.

Per-parameter LR groups. The workspace parameters (∼2M of ∼24M total) receive 0.25× the base learning rate (5×10^{-5} vs 2×10^{-4} for the backbone). This is motivated by the observation that the workspace creates a sharper loss landscape: at equal LR, workspace gradients dominate and cause instability. The backbone LR is kept at 2×10^{-4} to match Cell E (the control), preserving the clean C-vs-E comparison. This is analogous to the standard practice of using different LR groups for different module types in fine-tuning.

Four stabilization techniques. The workspace and recurrent core introduce four sources of gradient instability that are not present in the baseline hybrid model. We address each with a targeted normalization:

1. **Per-parameter LR groups** (above): workspace parameters receive 0.25× the base LR, preventing the workspace from dominating the gradient.
2. $1/\sqrt{K}$ **residual scaling** (Section 4.3): scales the recurrent core's blend factor by $1/\sqrt{K}$, normalizing gradient accumulation across K iterations so that the total gradient norm is independent of K.
3. **Slot parameter normalization** (Section 4.2): constrains the learned slot parameters to unit RMS after each optimizer step, breaking the positive feedback loop where large slots → sharp attention → large gradients → larger slots.
4. **Spectral normalization** (Section 4.2): caps the spectral norm of all eight workspace weight matrices at $C = 5$ after each optimizer step, addressing the root cause of unbounded weight growth directly while preserving sufficient attention selectivity (max ratio ∼164:1).

Each technique targets a distinct mechanism: LR groups address the workspace's sharper loss landscape, $1/\sqrt{K}$ scaling addresses recurrent gradient accumulation, slot normalization addresses unbounded slot parameter growth, and spectral normalization addresses unbounded weight matrix growth. All four are deterministic (one hyperparameter: the spectral norm bound $C = 5$) and can be disabled independently for ablation.

Attention regularizers. In addition to the four stabilization techniques, two regularizers address a distinct problem: workspace *degeneracy* (the workspace acting as a global average pool rather than selective memory). The attention entropy penalty ($\lambda_{\text{ent}} = 0.01$) minimizes the mean entropy of read/write attention distributions, pushing toward selectivity. The slot diversity penalty ($\lambda_{\text{div}} = 0.01$) minimizes the mean cosine similarity between slots, pushing them to carry different information. Both are task-agnostic and remain active during language training. Config: `ws_entropy_weight`, `ws_diversity_weight`.

Architectural stability: zero-init gates and slot decay. The external constraints and regularizers address symptoms of instability but not the root cause: the workspace's feedback loop is amplifying rather than contractive. Two architectural changes make the workspace stable by construction. **Zero-initialized gates**: gate parameters are initialized to -2 ($\sigma(-2) \approx 0.12$), so the workspace starts with a small contribution and the feedback loop is mostly inactive. A gate-specific LR group (10× workspace LR) ensures the gates open within the training budget. **Slot decay**: the slot update includes a learned decay factor γ (initialized at 1.0): $\text{slots} = \text{norm}(\gamma \cdot \text{slots} + g_{\text{read}} \cdot \text{read\_out})$. The decay provides a restoring force—old information

fades unless actively reinforced—making the feedback loop contractive. Config: `gate_init` (default −2), `slot_decay_init` (default 1.0).

6.3 Synthetic Tasks

All tasks are character-level, generated on-the-fly (infinite data, no overfitting), and automatically verifiable. Each has a **depth knob** that controls how many reasoning steps a correct answer requires.

Task 1: Chained assignment arithmetic (multi-hop composition). Variables are defined in terms of previous variables, all arithmetic mod 97:

```
a=7;b=a*3+2;c=b-a;d=c*2;?d;
```

The model must follow a chain of k dependencies. Distractor variables are inserted at random positions. This is the core multi-hop reasoning task.

Task 2: Permutation tracking (state-tracking). n items undergo k swap operations; the model must report one item's final position:

```
n=6;2,5;0,3;5,1;?3;
```

State-tracking is a documented weakness of linear SSMs (Arora et al., 2024); this task tests whether the workspace compensates for the architecture's known gap.

Task 3: Single-hop recall (control). Shallow lookup with distractors:

```
a=5;b=12;c=8;d=3;e=15;?c;
```

This is the control for selective ablation: the J-SPACE signature requires that killing the workspace leaves this intact while Tasks 1–2 collapse.

Training uses depths 2–8; evaluation extends to 2–16, measuring extrapolation beyond the training distribution.

6.4 Pre-Registered Results

Four measurements constitute proof, in ascending order of importance:

R1 — Capability:	Cell D beats Cell B by ≥10 accuracy points on Tasks 1–2 at depths 10–16 (beyond training range).
R2 — Compute scaling:	Accuracy on deep problems increases monotonically with K; harder depths need higher K to saturate.
R3 — Workspace decodability:	Linear probes on workspace slots can decode intermediate variable values, with decodability rising across loop iterations. This is the J-lens analogue.
R4 — Selective ablation:	Replacing workspace slots with their training-set mean causes Tasks 1–2 to collapse (≥30pt drop) while Task 3 barely moves (≤5pt drop)—mirroring Anthropic's J-SPACE finding.

R3 and R4 are what make this a *J-SPACE proof* rather than just another architecture ablation. R4 in particular tests the causal claim: the workspace is not merely correlated with reasoning performance, it is *causally responsible* for it.

7 Preliminary Results

Training is currently in progress on Tenstorrent Quietbox 2 hardware (Blackhole architecture, bfloat16-native). An automated evaluation loop monitors all cells, running `eval_ttnn.py` on each new checkpoint (every 25 steps) and tracking Task 1 accuracy for plateau detection. The training budget is 3,000 steps per cell (∼2 days for Cell C at ∼1 min/step, ∼4 days for Cell D at ∼2 min/step). The following preliminary observations cover the first ∼700 training steps of the control cell (E) and the progression of workspace cells (C and D) through multiple stabilization iterations: external constraints (4 techniques), regularizers (entropy + diversity), and finally the architectural fix (zero-init gates + slot decay).

Cell B has plateaued on Task 1. Cell B (hybrid baseline, lr=6×10^{-4}) was evaluated at steps 100, 200, 300, and 400. Task 1 (multi-hop chain reasoning) accuracy was 0% at all four evals—a confirmed plateau. The model masters Task 3 (single-hop recall, 100%) and performs well on Task 2 (permutation tracking, 80%), but cannot follow chains of variable dependencies. This is the expected result: without a workspace, the model has no mechanism to maintain intermediate values across a multi-hop chain. Cell B has been stopped after step 450 and its device reassigned to Cell E (the learning-rate control).

Cell E confirms the Task 1 plateau is architectural, not LR-related. Cell E (B architecture, C/D learning rate of 2×10^{-4}) has been evaluated at steps 100 through 700. Task 1 accuracy is 0–5% (chance level) across all evals—the same plateau as Cell B, despite using a 3× lower learning rate. This confirms that B's inability to do multi-hop reasoning is an architectural limitation, not a learning-rate artifact. E saturates at 62% overall (80% Task 2, 100% Task 3) by step 600, matching Cell B's plateau. The loss has been flat at ∼0.94 since step 400. This is the benchmark the workspace cells must beat.

Workspace gradient instability and the four stabilization techniques. Initial Cell C training (without stabilization) collapsed at step 200 when the learning rate reached its peak of 2×10^{-4}: the gradient norm spiked from ∼1.3 to ∼13.4, and the model's output degenerated to a single token ("72") regardless of input. Cell E, which has no workspace, passed the same step without issue (grad norm 2.1), confirming the instability is workspace-specific.

We identified four distinct instability mechanisms and addressed each with a targeted normalization (Section 6.2):

1. **Workspace sharper loss landscape:** The workspace parameters dominate the gradient at equal LR. Per-parameter LR groups (workspace at 0.25×) delayed the collapse from step 200 to step 250 but did not prevent it.
2. **Recurrent gradient accumulation:** Cell D's gradient norm was ∼20–22 at early steps (vs ∼5 for C/E), consistent with $\sqrt{K}$ accumulation. The $1/\sqrt{K}$ residual scaling reduced D's gradient norm to ∼9.7—roughly half, matching the theoretical prediction.
3. **Unbounded slot growth:** The slot parameters grow via a positive feedback loop (large slots → sharp attention → large gradients → larger slots). Slot parameter normalization constrains slots to unit RMS after each optimizer step. With this fix, Cell C passed step 200 (the original collapse point) with grad norm 1.0, but collapsed at step 300 with grad norm 7.3—the slot vectors were constrained, but the attention weight matrices continued to grow.
4. **Unbounded attention weight growth:** Even with slot magnitudes constrained, the W_Q and W_K projections can grow, producing sharp softmax distributions. We initially tried attention logit clamping ($C = 5$) to cap the resulting softmax sharpness, but this was a symptom-level fix—the weights continued to grow, and Cell D still showed escalating gradient spikes (10.8 → 13.7). Spectral normalization at bound $C = 1$ (the GAN default)

eliminated the early spikes and allowed both cells to pass their previous collapse points (Cell C at step 300, Cell D at step 250 with grad norm 4.2, vs 15.2 in the 3-fix run). However, the collapse re-emerged at peak LR (step 400 for C with grad norm 19.7, step 300 for D with grad norm 61.1). Analysis revealed a *capacity mismatch*: spectral norm 1 limits the attention ratio to $e^{2/\sqrt{d_h}} \approx 1.2$:1 (nearly uniform), so the workspace cannot provide the selective attention the backbone expects at peak LR. The backbone adapts fast (LR 2×10^{-4}), develops expectations that the workspace will provide selective attention, but the workspace is trapped at spectral norm 1 and cannot deliver—the resulting tug-of-war at the constraint boundary produces the gradient spike. The fix is to cap at $C = 5$ instead of 1, allowing attention ratios up to ∼164:1—enough for selective attention, but still bounded. The cap (rather than hard normalization to C) lets weights grow naturally up to the bound, preventing disruption when resuming from checkpoints saved with smaller spectral norms.

The first three stabilizations delayed the collapse (from step 200 to step 300 for Cell C) but did not prevent it. Spectral normalization at $C = 1$ passed the old collapse points but re-collapsed at peak LR due to the capacity mismatch. The current approach—spectral norm capped at $C = 5$—addresses the root cause while preserving sufficient attention capacity. Cells C and D have been resumed from their step 200 checkpoints with all four stabilizations applied and are currently training through the peak LR region.

Workspace degeneracy and attention regularizers. Even with stable training (no gradient collapse), diagnostic inspection of the workspace at step 100 revealed a *degeneracy* problem: the workspace was acting as a global average pool rather than a selective memory. Read attention entropy was 3.79 (out of $\ln 128 \approx 4.85$ for uniform), meaning slots attended nearly uniformly across all positions. Write attention entropy was 2.75 (out of $\ln 16 \approx 2.77$ for uniform), meaning every position wrote equally to every slot. The slot states did not change across recurrent core iterations. The workspace contained a blurred average of the entire sequence, not specific variable values.

This degeneracy arises because the cross-entropy loss provides only a diffuse gradient signal for workspace selectivity: the loss is several steps removed from the workspace's internal dynamics, and if the workspace isn't storing useful intermediates, the gradient for "you should have stored b" is weak. The model gets the same loss whether the workspace averages or selects, so there is no pressure to become selective.

We addressed this with two task-agnostic regularizers that can remain active during language training (unlike an auxiliary loss that would require known intermediate values):

1. **Attention entropy penalty** ($\lambda_{\text{ent}} = 0.01$): minimizes the mean entropy of read and write attention distributions, pushing the workspace toward selective (low-entropy) attention. This directly targets the observed uniformity without specifying *what* to attend to.
2. **Slot diversity penalty** ($\lambda_{\text{div}} = 0.01$): minimizes the mean cosine similarity between pairs of slots, pushing the 16 slots to carry different information rather than collapsing to identical averaged representations.

Both regularizers are computed via host-side PyTorch autograd on the cached attention and slot tensors, with gradients injected into the workspace backward pass at the softmax and slot-state boundaries. They contribute ∼5% of the total loss signal—a gentle nudge, not a dominant force.

Regularizers work but reveal a deeper architectural issue. With the entropy and diversity regularizers active (weight 0.01 each), Cell C's workspace underwent a dramatic transformation. By step 400, write attention entropy dropped from 2.75 to 0.224 (out of 2.77 for uniform)—positions were now selectively writing to specific slots. Read attention entropy dropped from 3.79 to 3.09. The workspace went from a global average pool to an actual selective memory. The loss dropped to 0.55 (vs ∼1.0 in previous runs), and the model began attempting

digit predictions instead of outputting EOS.

However, the model collapsed at step 450: loss jumped from 0.55 to 1.32, gradient norm spiked to 51.7. The entropy regularizer had pushed the attention to be sharp, but the sharpening exceeded what the spectral norm cap could stably support at peak LR. The regularizer proved the workspace *can* learn selective attention, but revealed that the instability is structural, not just a tuning problem.

Architectural stability: the feedback loop problem. Analysis of the collapse mechanism reveals a fundamental architectural issue. The workspace has a *feedback loop with gain*: it reads from x, updates slots, then writes from slots back to x. Each pass through this loop can amplify—sharp attention → large updates → changed representations → sharper attention. The external constraints (spectral norm, slot normalization, entropy regularization) cap individual components but do not make the loop itself contractive. This is structurally different from architectures that are stable by construction:

- **LSTMs**: the forget gate makes the cell state update contractive—the state can only change by a bounded amount per step.
- **ResNets/LoRA**: the residual branch is zero-initialized, so the module starts as identity and gradually learns to contribute.
- **Transformers**: each layer has residual + layer norm, and the attention output is bounded by the norm-constrained value vectors.

Our workspace has residual connections and norms, but two structural features prevent it from being naturally stable:

(1) Gates initialized at 0.5. The read and write gates use $\sigma(0) = 0.5$, so the workspace starts by modifying the state significantly rather than starting as identity. The feedback loop is active from step 0, before the model has learned to use it safely. This is the opposite of the zero-initialization principle that stabilizes additive modules.

(2) No forget/decay on slots. The slot update is $\text{slots} = \text{norm}(\text{slots} + g_{\text{read}} \cdot \text{read\_out})$—purely additive (plus normalization). There is no mechanism to decay or forget stored information. Once information enters a slot, it persists, and sharp attention keeps adding more. The feedback loop has no restoring force: if attention gets too sharp and overwrites a slot, nothing pulls it back.

Architectural fix: zero-init gates and slot decay. We address both issues with two structural changes that create a natural basin of attraction—a "tennis ball in a bucket" rather than a knife edge:

(1) Zero-initialized gates. Initialize the gate parameters to -2, giving $\sigma(-2) \approx 0.12$. The workspace starts with a small but non-trivial contribution: the feedback loop is mostly inactive. The gates gradually open as the model learns that the workspace is useful. A gate-specific LR group ($10\times$ the workspace LR) ensures the gates reach $\sigma(0) = 0.5$ within ∼1,000 steps, fitting the 3,000-step training budget. This is the same principle as zero-initialization in LoRA, tuned for a shorter training horizon.

(2) Slot decay. Change the slot update to $\text{slots} = \text{norm}(\gamma \cdot \text{slots} + g_{\text{read}} \cdot \text{read\_out})$ where γ is a learned scalar initialized at 1.0. The decay makes the slot update contractive: old information naturally fades unless the read gate actively reinforces it. The feedback loop now has a restoring force—if attention gets too sharp and overwrites a slot, the decay pulls it back toward the learned slot embedding. The model must learn to overcome the decay, which means it only uses the workspace when the gradient signal is strong enough to justify it.

Together, these make the workspace start stable (near-zero gates) and stay stable (decaying slots). The external constraints (spectral norm, slot normalization, regularizers) become less critical because the architecture itself is contractive rather than amplifying.

Backward pass verification. The workspace backward pass has been verified against PyTorch autograd on CPU. All gradients—input gradients, weight gradients (8 matrices), slot gradients, norm weights, and gate scalars—match to within float32 precision ($< 10^{-4}$ relative error). The gradient explosion was confirmed to be a training dynamics issue, not a backward pass bug.

Summary table. Table 4 shows the evaluation results collected so far by the automated eval loop.

8 Discussion

8.1 Why This Might Work

The J-SPACE finding tells us that reasoning in large models is carried by a small, privileged subspace. The TRM finding tells us that iteration through a small network can substitute for parameter count. The Mamba-2 coincidence tells us that the SSM's compressed state already has the functional shape of the J-SPACE. Jasper combines all three: the workspace provides the privileged subspace, the recurrent core provides the iteration, and the hybrid Mamba-2 backbone provides the substrate.

If this works, the implication is that we can build reasoning models that are:

- **Small enough for the browser:** ∼1B parameters, flat memory in both sequence length and loop count.
- **Controllable at inference:** K can be adjusted per problem, trading compute for accuracy.
- **Interpretable:** workspace probes (R3) provide a window into the model's reasoning, unlike opaque latent reasoning in recurrent transformers.
- **Verifiable:** selective ablation (R4) provides a causal test of whether the workspace is doing the reasoning, not just correlated with it.

8.2 Why It Might Not Work

Synthetic-task wins at 30M parameters have a real history of not transferring to language at scale. The tasks we use (arithmetic chains, permutation tracking) are structurally simple and may not capture the full complexity of language reasoning. The workspace may help on these tasks without generalizing to the kind of reasoning that matters for real applications.

The gradient instability was a significant concern early in development, but the stabilization journey has been progressively successful: each fix addressed a real, identified mechanism, and each pushed the collapse further out (step 200 → 300 → 400 with $C = 1$ → step 450 with $C = 5$ + regularizers). The progression from external constraints (LR groups, spectral norm, slot normalization) to regularizers (entropy, diversity) to architectural changes (zero-init gates, slot decay) reflects a deepening understanding: the instability is not a tuning problem but a structural property of the feedback loop. The architectural fix—zero-initialized gates ($\sigma(-2) \approx 0.12$) and slot decay—makes the workspace contractive by construction, creating a natural basin of attraction rather than a knife edge. A gate-specific LR group (10× workspace LR) ensures the gates reach $\sigma(0) = 0.5$ by step ∼1,000 within the 3,000-step budget. The remaining risk is whether the contractive architecture is too conservative: if the decay prevents the workspace from maintaining information long enough for multi-hop chains, or if the gates open too slowly to contribute meaningfully within the training budget. Both parameters are tunable (gate init value, decay rate, gate LR multiplier), and the zero-init principle is proven in LoRA and related work.

The more substantive question is whether the workspace architecture provides enough *computational leverage* to matter at scale. The current 30M-parameter model shows promising

signs: with the regularizers, the workspace learned selective attention (write entropy dropping from 2.75 to 0.224) and the loss dropped to 0.55 (vs ∼1.0 in previous runs), before the architectural instability caused collapse. This confirms the workspace *can* learn to be selective and *can* improve the loss—the question is whether the architectural fix allows this learning to persist stably. If so, the workspace cells should begin showing Task 1 accuracy as training progresses; if not, a larger workspace (more slots, wider projections) or an auxiliary loss (using known intermediate values from the synthetic tasks) may be needed.

Finally, the Mamba-2 coincidence may be more metaphorical than functional. The SSM state and the J-SPACE have similar shapes, but they operate at different levels of abstraction—the SSM state is a low-level sequence summary, while the J-SPACE is a high-level conceptual workspace. Whether the workspace module can bridge this gap is an empirical question.

8.3 Decision Framework

The proof-of-concept is designed to produce a clear go/no-go decision before any significant cloud spend:

- **Go:** R1 and R2 pass, and at least one of R3/R4 shows the workspace doing real causal work → fund a 300M language-scale ablation (∼$3–5K).
- **Pivot:** R1 fails but Cell B's efficiency story stands → keep the hybrid backbone, drop the workspace, proceed on hybrid distillation.
- **Kill:** Cell D ≤ Cell B everywhere → the novel architecture track is dead; fall back to sampled-attempts-plus-verifier on a standard model.

The entire proof-of-concept costs ∼$20–30 of electricity and under a week of compute. This buys the right to spend $5K intelligently, not the conclusion itself.

9 Related Work

Global workspace in LLMs. Anthropic (2026) discovered the J-SPACE in Claude Opus 4.6 using the Jacobian lens, showing that a small set of verbalizable representations functions as a global workspace (Baars, 2005; Dehaene, 2014). Our workspace module is an explicit engineering of this concept.

Iterative and recurrent reasoning. Jolicoeur-Martineau (2025) showed that tiny recursive networks (7M parameters) can outperform much larger models on ARC-AGI through iterative refinement. Geiping et al. (2025) scaled recurrent-depth transformers to 3.5B parameters, demonstrating test-time compute scaling in latent space. Gehring et al. (2024) explored relaxed recursive transformers with shared parameters. Our recurrent core follows the TRM paradigm but uses a hybrid Mamba-2/attention backbone and an explicit workspace state.

Hybrid SSM-attention models. Dao and Gu (2024) established the SSD framework connecting SSMs and attention. Wang et al. (2025) showed that hybrid Mamba reasoning models can match transformer-based R1-distilled models on AIME/MATH while generating 3× faster. NVIDIA's Nemotron line has repeatedly shown hybrid models matching or beating same-size transformers on long reasoning traces. Our architecture follows the hybrid pattern but adds the workspace and recurrent core.

Test-time compute scaling. The broader literature on scaling test-time compute—from self-consistency (Wang et al., 2023) to process reward models (Lightman et al., 2023) to OpenAI's o1/o3 models—operates in token space. Our approach scales test-time compute in latent space, which is more memory-efficient but less legible.

10 Conclusion

We have presented Jasper, an architecture that combines three recent insights—the J-SPACE as a compact reasoning workspace, iterative refinement as a substitute for parameter count, and Mamba-2's SSM state as a native substrate for the workspace—into a single system. The architecture reasons in latent space without generating tokens, keeps memory flat in both sequence length and loop count, and provides interpretable and causally testable reasoning through workspace probes and selective ablation.

The proof-of-concept experiment is designed to produce a clear verdict at minimal cost: if the workspace causally carries multi-step reasoning (R4) and the recurrent core provides test-time compute scaling (R2), the architecture warrants scaling to language-scale experiments. If not, the hybrid Mamba-attention backbone still stands as an efficient baseline for in-browser deployment.

The broader claim is that reasoning in language models need not be tied to token generation. The brain reasons through recurrent firing patterns before articulating a thought; perhaps language models should too.

References

Wes Gurnee and Jack Lindsey et al. Verbalizable representations form a global workspace in language models. *arXiv preprint arXiv:2607.15495*, 2026.

Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Alexandre Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025. [Samsung SAIT Montreal]

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, pages 10041–10071, 2024.

Bernard J. Baars. Global workspace theory of consciousness: Toward a neuroscience of human experience. *Progress in Brain Research*, 150:45–53, 2005.

Stanislas Dehaene. *Consciousness and the Brain: Deciphering How the Brain Codes Our Thoughts*. Viking, 2014.

Junxiong Wang et al. M1: Towards scalable test-time compute with Mamba reasoning models. *arXiv preprint*, 2025.

Simran Arora, Sabhash Rao, and Christopher Ré. Theoretical limitations of self-attention and state space models in reasoning tasks. *arXiv preprint*, 2024.

Jonas Gehring, Kilian Golde, and David Grangier. Relaxed recursive transformers. *arXiv preprint arXiv:2410.10672*, 2024. [Google DeepMind]

Xuezhi Wang et al. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Hunter Lightman et al. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. [OpenAI]

Hongyu Wang et al. DeepNet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022. [Microsoft]

Cell	Step	Overall	Task 1	Task 2	Task 3
A	200	30%	0%	80%	10%
A	300	35%	0%	75%	30%
B	100	22%	0%	60%	5%
B	200	30%	5%	80%	5%
B	300	57%	0%	75%	95%
B	400	60%	0%	80%	100%
E	100	22%	0%	60%	5%
E	200	32%	0%	75%	20%
E	300	28%	0%	70%	15%
E	400	38%	0%	75%	40%
E	500	40%	0%	75%	45%
E	600	62%	5%	80%	100%
Old runs (no stabilization, collapsed at step 200):					
C (old)	100	23%	0%	60%	10%
C (old)	200	7%	5%	15%	0%
D (old)	100	5%	0%	15%	0%
D (old)	200	22%	0%	60%	5%
With LR groups + slot norm + $1/\sqrt{K}$ (collapsed at step 300):					
C (3-fix)	300	25%	0%	70%	5%
C (3-fix)	400	23%	0%	65%	5%
C (3-fix)	500	20%	0%	60%	0%
C (3-fix)	600	15%	0%	45%	0%
D (3-fix)	200	22%	0%	60%	5%
With LR groups + slot norm + $1/\sqrt{K}$ + spectral norm $C = 1$ (passed old collapse, re-collapsed at peak LR):					
C (C=1)	300	33%	0%	85%	15%
D (C=1)	re-collapsed at step 300 (grad norm 61)				
With all 4 stabilizations, spectral norm capped at $C = 5$ + regularizers (collapsed at step 450):					
C (reg)	400	—	—	—	—
C (reg)	loss dropped to 0.55 (best ever), write entropy 2.75 → 0.224, then collapsed at step 450				
With architectural fix (zero-init gates + slot decay, fresh start, in progress):					
C (arch)	training, loss 4.90 → 1.96 in 50 steps (pre-reboot), stable				
D (arch)	training, loss 4.94 → 3.31 in 38 steps (pre-reboot), stable				

Table 4: Evaluation results from the automated eval loop (5 examples per task per depth, depths 2/4/6/8). Task 1 is multi-hop chain reasoning—the target metric for workspace benefit. Both B and E show 0% Task 1 across all evals (E's 5% at step 600 is likely noise—1/20 examples), confirming the architectural plateau. The old C run collapsed at step 200; with 3 stabilizations it passed step 200 but collapsed at step 300. Spectral norm at $C = 1$ passed the old collapse points (C at step 300: 33% overall, 85% Task 2—best ever for C) but re-collapsed at peak LR due to the capacity mismatch. With $C = 5$ + regularizers, the workspace learned selective attention (write entropy 2.75 → 0.224) and the loss dropped to 0.55, but the model collapsed at step 450 due to the structural feedback loop instability. The architectural fix (zero-init gates + slot decay) is currently training with a 3,000-step budget.